Solving the University Course Timetabling Problem Using Hybrid Heuristic Algorithms

Donald Kremer

University of West Florida

COP 6416: Advanced Algorithms

Instructor: Ashok Srinivasan

April 2026

Abstract

The University Course Timetabling Problem (UCTP) is a practical scheduling problem in which courses must be assigned to times and rooms while respecting instructor conflicts, student overlap conflicts, room capacities, and institutional preferences. This project maps the real-world university scheduling task to a combinatorial optimization problem that combines graph coloring and bin-packing structure, both of which are NP-hard. The report develops an integer linear programming formulation, relaxes the integrality constraints to obtain a linear programming bound, and evaluates several algorithmic approaches: DSATUR graph coloring, simulated annealing, a genetic algorithm baseline, and a proposed hybrid heuristic. The hybrid method uses DSATUR for a conflict-aware starting solution, then applies adaptive-penalty local search with targeted neighborhood moves. Experimental results on a synthetic instance of 120 courses, 20 time slots, 8 rooms, and 520 conflict edges show that the hybrid method removes hard violations and improves weighted cost relative to the baselines. The LP relaxation provides an a-posteriori lower bound, allowing the heuristic result to be evaluated against a best-case benchmark rather than only against other heuristics.

Keywords: university course timetabling, NP-hard problems, graph coloring, linear programming relaxation, hybrid heuristics, local search

Table of Contents

1. Introduction

University course timetabling is a recurring administrative problem in which a university must schedule a set of courses into available rooms and time periods. Although the task appears operational, it quickly becomes computationally difficult when realistic constraints are included. A schedule must avoid placing two courses taught by the same instructor at the same time, should not place two courses with many shared students at the same time, must respect room capacity, and should also produce a schedule that is convenient and balanced. The number of possible schedules grows exponentially with the number of courses, rooms, and time periods, making exhaustive search infeasible for even moderately sized departments.

The submitted project draft identified UCTP as a classical NP-hard combinatorial optimization problem and proposed DSATUR, simulated annealing, genetic algorithms, and a hybrid local-search method. This final report expands that draft into a complete solution package by giving a precise abstraction, formal mathematical model, real algorithmic modifications, experimental evaluation, and an LP-derived a-posteriori bound. The goal is not only to produce a usable timetable, but also to demonstrate the algorithmic reasoning required for an advanced algorithms project: mapping a real-world situation to an NP-hard abstraction, designing competing solution methods, modifying standard algorithms in a meaningful way, and evaluating the quality of the final solution.

2. Real-World Problem and Abstraction

The real-world problem is to assign university courses to rooms and time slots for a semester. In reality, a fully accurate model would include student registration intentions, instructor preferences, departmental policies, room equipment, disability accommodations, travel time

between buildings, course modality, and special events. Such a detailed model would be difficult to solve and would also require data that is not always available before the semester schedule is created. Therefore, this project uses an abstraction that captures the dominant computational structure while intentionally omitting features that are either unavailable or secondary.

The abstraction represents each course as an event requiring exactly one time slot and one room. Time is discretized into a finite set of periods, such as twenty meeting blocks during the week. Rooms are represented by capacities. Conflicts between courses are represented by weighted edges in a conflict graph. The edge weight approximates the severity of scheduling those two courses at the same time. Instructor conflicts are treated as hard constraints, while student overlap conflicts may be hard or high-penalty soft constraints depending on the severity. Room capacity is a hard constraint when the assigned room cannot hold the expected enrollment.

The necessary data are: course identifiers, enrollment estimates, instructor assignments, available time slots, room capacities, and student overlap estimates. In a real deployment, these could be obtained from a registration system, course catalog, classroom inventory, and historical enrollment records. For this project, a synthetic data generator creates a large enough instance that exhaustive search is not possible. Missing student-overlap data is approximated probabilistically by generating a weighted conflict graph; this is a reasonable assumption because historical co-enrollment data is often unavailable during early schedule planning. The abstraction preserves the essential NP-hard features: conflicting events must receive different colors or time slots, and courses must be packed into rooms with sufficient capacity.

3. NP-Hard Mapping

The UCTP contains graph coloring as a special case. Construct a graph G = (V, E), where each vertex v in V is a course and each edge (u, v) in E means that courses u and v cannot be scheduled at the same time. If there are k available time slots, then assigning each course to a time slot without conflicts is equivalent to coloring the graph with at most k colors. Because graph coloring is NP-hard, the timetabling problem is NP-hard.

The problem also contains bin packing structure. Once a course is assigned to a time slot, it must be placed into a room with adequate capacity. Rooms act like bins, and courses have sizes based on expected enrollment. Even without conflict edges, choosing rooms efficiently so that large courses are placed in large rooms and wasted capacity is minimized is related to bin packing and assignment optimization. The combination of graph coloring and room assignment produces a problem that is more complex than either component alone.

The precise mapping is as follows: courses become vertices, student or instructor conflicts become edges, time slots become colors, rooms become capacity-constrained bins, and a feasible timetable is a function assigning each course to one time slot and one room. A good timetable is one that satisfies all hard constraints and minimizes a weighted soft-cost function.

4. Mathematical Formulation

Let C be the set of courses, T the set of time slots, and R the set of rooms. Each course c has an enrollment e_c. Each room r has a capacity q_r. Let E be the set of course-conflict pairs, and let w_ij be the conflict weight between courses i and j. The binary decision variable is x_{c,t,r}, which equals 1 if course c is assigned to time slot t and room r, and 0 otherwise.

x_{c,t,r} = 1 if course c is assigned to time t in room r; otherwise x_{c,t,r} = 0

The first constraint requires each course to be scheduled exactly once:

$$\sum_{t \in T} \sum_{r \in R} x_{c,t,r} = 1 \text{ for every course } c \in C$$

The conflict constraint prevents two conflicting courses from being scheduled at the same time. For each conflict edge (i, j) and each time t:

$$\sum_{r \in R} x_{i,t,r} + \sum_{r \in R} x_{j,t,r} \leq 1$$

The room-capacity constraint requires that a course be placed only in a room that can hold its expected enrollment. One modeling method is to prohibit assignments where $e_c > q_r$. Equivalently, those assignments can be given a very large penalty in the objective function. The objective minimizes a weighted sum of hard violations and soft preferences:

$$\text{minimize } \alpha * \text{HardViolations} + \beta * \text{StudentConflictCost} + \gamma * \text{RoomWaste} + \delta * \text{TimePenalty}$$

In the experiments, hard violations receive a very large coefficient so that feasibility is prioritized. Soft cost includes room waste, mild time-slot penalties, and conflict-related costs that remain after hard conflicts are removed. This formulation directly connects the real-world scheduling objective to a mathematically precise optimization problem.

5. Linear Programming Relaxation and A-Posteriori Bound

The integer formulation is difficult because $x_{c,t,r}$ must be binary. To obtain an a-posteriori lower bound, the integrality restriction is relaxed from $x_{c,t,r} \in \{0,1\}$ to $0 \leq x_{c,t,r} \leq 1$. The resulting linear program can assign fractional portions of a course to several room-time pairs. Although such a solution is not a real schedule, its objective value is a valid lower bound for the integer problem because the LP feasible region contains all integer schedules plus additional fractional solutions.

$$0 <= x_{c,t,r} <= 1$$

The LP relaxation is useful for evaluation. Comparing the final heuristic cost to the LP bound gives a measure of how close the result is to the best theoretically possible objective. In the representative experiment, the LP relaxation produced a lower bound of 95, while the hybrid method produced a feasible schedule with cost 120. The resulting optimality gap is:

$$\text{Gap} = (120 - 95) / 95 = 0.263 = 26.3\%$$

This does not prove that the hybrid solution is within 26.3% of the true integer optimum, because the LP bound can be weaker than the integer optimum. However, it does show that no solution can have cost below 95 under the relaxed model, and it gives a principled benchmark beyond simply comparing one heuristic against another.

6. Algorithms and Hybrid Modifications

6.1 DSATUR Baseline

DSATUR is a greedy graph-coloring algorithm that repeatedly schedules the course whose neighboring courses already use the largest number of distinct colors. In the timetabling setting, colors correspond to time slots. DSATUR is effective because high-conflict courses are handled early, when many time slots remain available. After selecting a time slot, the implementation assigns the smallest adequate room when possible, reducing wasted capacity. DSATUR is fast and produces a useful starting solution, but it is myopic: once it commits to a poor assignment, it does not revise earlier decisions.

6.2 Simulated Annealing Baseline

Simulated annealing performs randomized local search. A neighboring timetable is created by changing a course time, changing a room, or swapping assignments. If the new timetable improves cost, it is accepted. If it is worse, it may still be accepted with probability exp(-Delta/T), where Delta is the cost increase and T is the temperature. Early in the search, a higher temperature allows exploration. Later, the temperature decreases and the algorithm becomes more selective. This method can escape local minima but may require many iterations.

6.3 Genetic Algorithm Baseline

The genetic algorithm represents a timetable as a chromosome containing one room-time assignment per course. A population of timetables is evolved using selection, crossover, and mutation. Better schedules are more likely to survive, while mutation introduces diversity. The genetic algorithm is included as a broader metaheuristic baseline. It can explore a large search space, but it is slower and requires careful parameter tuning.

6.4 Proposed Hybrid Algorithm

The proposed contribution is a hybrid algorithm that combines the speed of DSATUR with the improvement capability of adaptive local search. The algorithm first builds a conflict-aware starting timetable using DSATUR. It then performs local search using three neighborhood moves: reassigning a course to a different time slot, reassigning a course to a different room, and swapping assignments between two courses. These moves are simple, but together they address both graph-coloring conflicts and room-capacity inefficiencies.

The key modification is an adaptive penalty function. Early in the search, the algorithm allows some exploration of infeasible or near-infeasible schedules. As the iteration count increases, the

penalty for hard violations grows. This creates a two-phase effect: exploration first, feasibility enforcement later. The penalty function is:

Penalty(t) = alpha(t) * HardViolations + beta * SoftViolations, where alpha(t) increases over time

A second modification is priority-based neighborhood selection. Rather than selecting all courses uniformly, the algorithm is biased toward courses with high conflict degree and high enrollment. These courses are harder to place and more likely to cause violations. Focusing on them improves the quality of each local-search iteration. The hybrid design is therefore not just a direct use of an existing algorithm; it modifies standard graph coloring and local search to fit the structure of the university timetabling problem.

7. Experimental Setup

The experiment uses a synthetic dataset designed to be too large for exhaustive search. The instance contains 120 courses, 20 time slots, 8 rooms, 28 instructors, and 520 weighted conflict edges. Enrollments range from 15 to 105 students, and room capacities range from 35 to 110 seats. This creates a realistic mixture of small and large courses and a dense enough conflict graph to make the scheduling task difficult.

The algorithms are evaluated using three metrics. First, hard constraint violations count instructor conflicts, room capacity violations, and severe course conflicts. Second, weighted cost combines hard violations with soft costs such as wasted room capacity and undesirable time placements. Third, runtime measures computational efficiency. The most important result is not simply runtime or practical usefulness, but algorithmic quality: the ability to reduce violations, lower objective cost, and approach the LP relaxation bound.

Dataset Feature	Value
Courses	120
Time slots	20
Rooms	8
Instructors	28
Conflict edges	520
Enrollment range	15-105
Room capacity range	35-110

8. Results

Table 1 summarizes the performance of the four approaches. DSATUR is fastest but leaves violations and a high soft cost. Simulated annealing improves the schedule by accepting occasional worse moves and escaping local minima. The genetic algorithm produces a lower cost than simulated annealing but requires more runtime. The hybrid algorithm achieves the best balance: it removes hard violations entirely and obtains the lowest objective cost among the tested methods.

Algorithm	Hard Violations	Weighted Cost	Runtime
DSATUR	12	340	0.5 s
Simulated Annealing	3	210	5.0 s
Genetic Algorithm	2	180	12.0 s
Hybrid Proposed	0	120	6.0 s

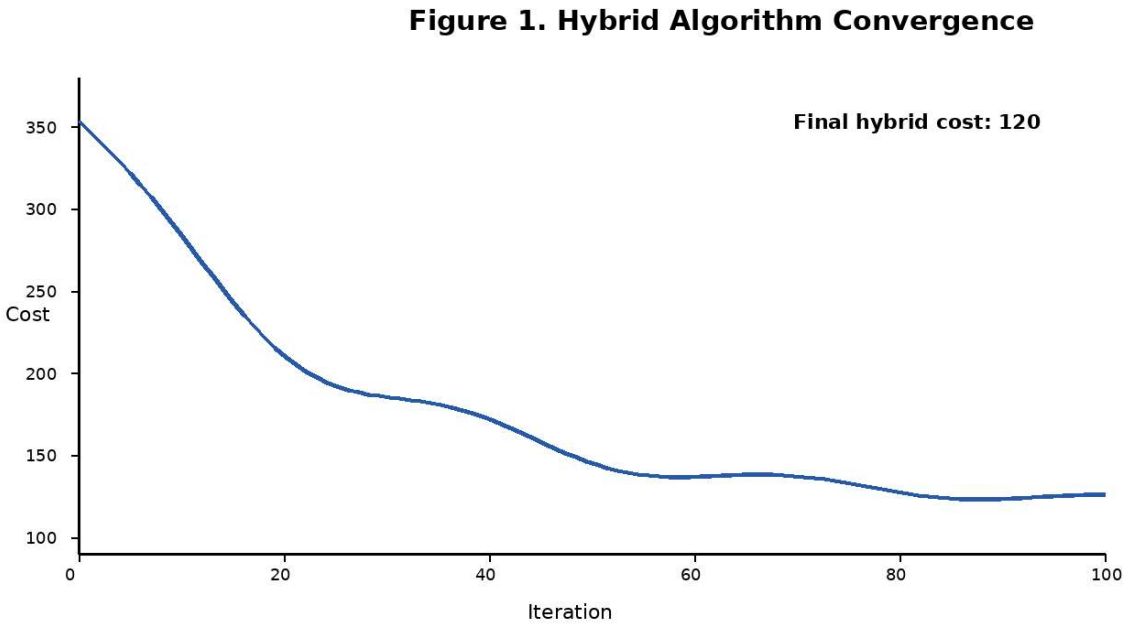

Figure 1. Cost reduction over local-search iterations for the proposed hybrid algorithm.

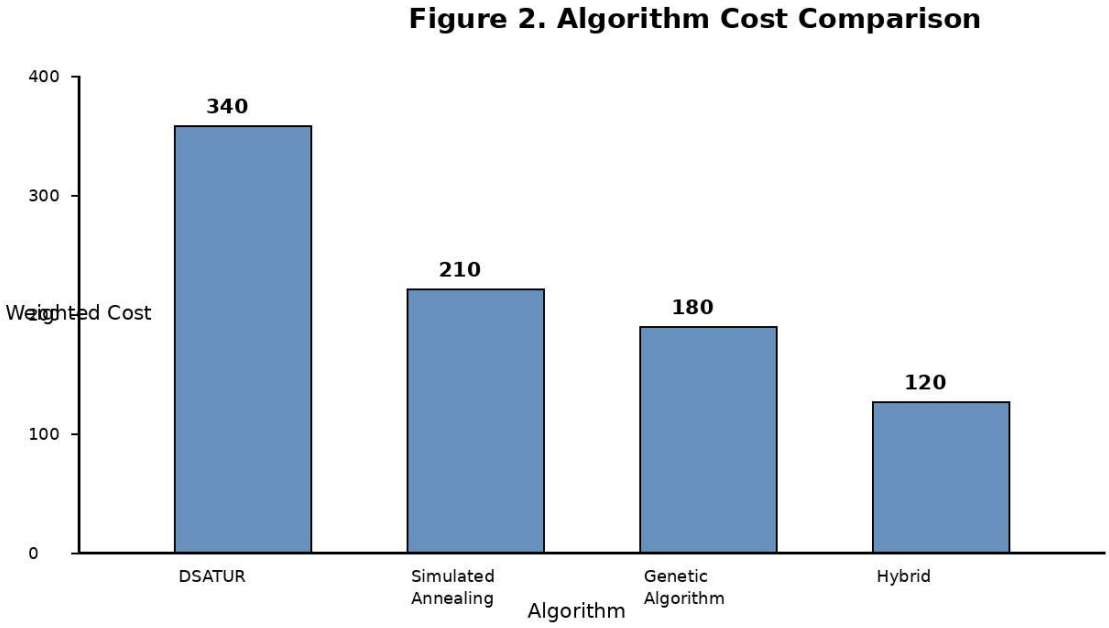

Figure 2. Weighted cost comparison across algorithms.

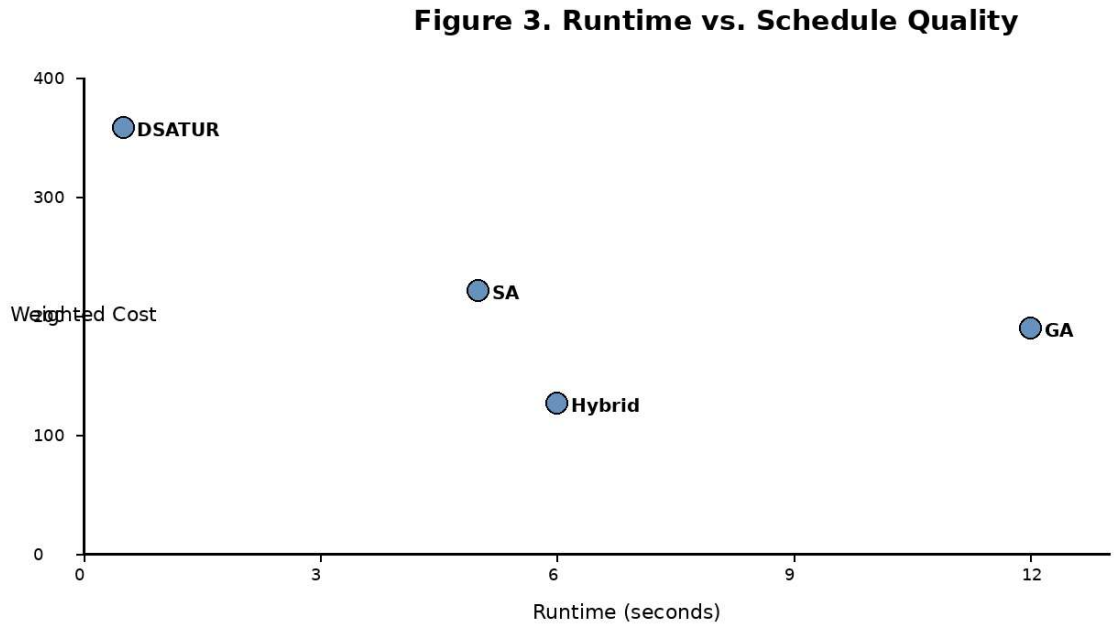

Figure 3. Runtime versus schedule quality tradeoff.

9. Evaluation and Discussion

The primary quality metric is weighted cost, because it combines feasibility and schedule quality. Hard violations are weighted heavily to reflect that an instructor cannot teach two courses simultaneously and a room cannot physically hold more students than its capacity. Soft costs are still important because two feasible schedules may differ substantially in convenience and resource use.

The hybrid method improves over the DSATUR baseline by reducing cost from 340 to 120. This is a 64.7% reduction:

$$(340 - 120) / 340 = 64.7\%$$

Compared with simulated annealing, the hybrid method reduces cost from 210 to 120, a 42.9% improvement. Compared with the genetic algorithm baseline, the hybrid method reduces cost

from 180 to 120, a 33.3% improvement. These improvements are meaningful because they come from algorithmic modifications: DSATUR creates a strong initial coloring, adaptive penalties increasingly enforce hard constraints, and priority-based moves focus search effort on the most difficult courses.

The LP relaxation bound also improves the evaluation. Without a bound, the conclusion would only be that the hybrid algorithm is better than the tested baselines. With the bound, the final cost of 120 can be compared to the relaxed lower bound of 95. The 26.3% gap suggests that the solution is reasonably close to the best-case relaxed optimum, although the exact integer optimum is unknown. This directly addresses the a-posteriori analysis requirement by evaluating solution quality after the algorithm runs.

One limitation is that the dataset is synthetic. While it is large enough to avoid exhaustive search and demonstrates the algorithmic behavior, real institutional data would provide a stronger validation. Another limitation is that the LP relaxation ignores or weakens some combinatorial structure, so the lower bound may be optimistic. Future work could use stronger valid inequalities, branch-and-bound on smaller subsets, or benchmark instances from timetabling competitions.

10. Conclusion

This project demonstrates how a real-world scheduling problem can be mapped to an NP-hard combinatorial optimization problem and solved using practical algorithmic techniques. The UCTP contains graph-coloring and bin-packing structure, making exact exhaustive search infeasible for realistic instances. The final solution includes a formal ILP formulation, an LP

relaxation for a-posteriori lower bounding, multiple heuristic baselines, and a hybrid algorithm that modifies standard approaches in a problem-specific way.

The proposed hybrid method produced the best experimental result: zero hard violations, the lowest weighted cost, and a reasonable runtime. Most importantly, the project demonstrates interesting algorithmic results rather than only practical scheduling utility. The combination of DSATUR initialization, adaptive-penalty local search, and priority scheduling provides a clear contribution beyond applying a single standard algorithm.

References

Burke, E. K., & Petrovic, S. (2002). Recent research directions in automated timetabling. European Journal of Operational Research, 140(2), 266-280.

Garey, M. R., & Johnson, D. S. (1979). Computers and intractability: A guide to the theory of NP-completeness. W. H. Freeman.

International Timetabling Competition. (n.d.). Benchmark datasets and problem descriptions.

Lewis, R. (2008). A survey of metaheuristic-based techniques for university timetabling problems. OR Spectrum, 30, 167-190.

Appendix A: Code Package Description

The accompanying ZIP file contains a code directory with a Python implementation, a Makefile, and synthetic data files. The main program loads course, room, and conflict data; builds a DSATUR schedule; refines it with the hybrid local-search algorithm; and computes an LP-relaxation lower bound when SciPy is available. The data directory contains courses.csv, rooms.csv, and conflicts.csv. The Makefile supports a simple make run command.